

Lab 4

Instructions for Lab Submission:

1. File Naming and Submission:
 - a. Submit your solution as a zip file named **lab4_<ERP>.zip**, where **<ERP>** is your ERP number. For example, if your ERP number is *12345*, the file should be named *lab4_12345.zip*.
 - b. Ensure that the zip file contains:
 - i. C++ source files named as **task<number>.cpp** (e.g., **task1.cpp**).
 - ii. No unnecessary files (such as executables, temporary files, or editor configs).
 - c. Do not submit **TEXT(.txt)** files.
2. Code Organization:
 - a. Write each task in its own **.cpp** file (as specified above).
 - b. Ensure that your code is modular and logically separated into functions when specified.
3. Code Neatness:
 - a. Ensure your code is well-formatted and easy to read. Use proper indentation and meaningful variable names.
 - b. Include appropriate comments to explain the logic where necessary (especially for functions and complex sections of the code).
 - c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
4. Code Compilation and Functionality:
 - a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
 - b. Double-check that the functionality of the program meets the requirements outlined in the task.
5. Additional Instructions:
 - a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
 - b. Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
 - c. If you encounter any issues or require clarifications, reach out before the submission deadline.
6. Deadline:
 - a. Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted.
 - b. Double-check that the zip file is correctly named and contains all necessary files before submitting.
7. Plagiarism Policy:
 - a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lab Tasks

Task 1: Fibonacci Series

Write a program that generates and prints the Fibonacci series up to N terms, where N is entered by the user. Use a **for** loop to calculate the terms in the sequence.

Task 2: Sum and Count Positive Numbers

When working with input-driven programs, it is common to process a series of numbers until a special "sentinel value" is entered. A sentinel value is a specific input that signals the program to stop reading further input. For example, entering -1 can indicate the end of input.

This problem focuses on practicing the use of a **while** loop, sentinel values, and basic accumulation logic. The goal is to collect and analyze user input until the stopping condition is met.

Write a C++ program that repeatedly takes integer input from the user until they enter **-1**. The program should:

1. Add up all the positive numbers entered.
2. Count how many positive numbers were entered.
3. Stop when the user enters **-1**.

Display the total sum, and the count of positive numbers entered.

Task 3: Sudoku Row Validator

Sudoku is a popular logic-based number puzzle. A standard Sudoku grid consists of 9 rows and 9 columns, and the rules require that each row, column, and 3×3 sub grid contain all digits from 1 to 9 exactly once. Ensuring that no number repeats within a row is one of the fundamental checks in validating a Sudoku solution. This task focuses on validating a single row.

Write a C++ program that takes 9 integers (each between 1 and 9) as input and checks whether they form a valid Sudoku row. A valid row must satisfy the following conditions:

1. Each number must be between 1 and 9.
2. No number should appear more than once.

Use a **for** loop to read the numbers and validate the row as they are entered.

```
fall25/Lab_Solutions/lab04 on ⌘ main  
> g++ task3.cpp && ./a.out  
Enter number 1: 1  
Enter number 2: 9  
Enter number 3: 2  
Enter number 4: 8  
Enter number 5: 3  
Enter number 6: 7  
Enter number 7: 4  
Enter number 8: 6  
Enter number 9: 5  
Valid Sudoku row.
```

```
fall25/Lab_Solutions/lab04 on ⌘ main  
> g++ task3.cpp && ./a.out  
Enter number 1: 1  
Enter number 2: 9  
Enter number 3: 2  
Enter number 4: 8  
Enter number 5: 3  
Enter number 6: 9  
Invalid row. Number 9 is repeated.
```

Task 4: Perceptron

In **machine learning**, a perceptron is one of the simplest models of an artificial neuron. It takes an input, multiplies it by a weight, adds a bias, and produces an output. Although modern neural networks use more advanced structures, the perceptron is important because it forms the foundation of how learning systems adjust weights to minimize errors.

Training a perceptron involves making guesses, comparing them with the correct outputs, calculating the error (called *loss*), and then adjusting the weight and bias to improve accuracy. This process is repeated until the error becomes very small.

In this task, we will simulate the training of a perceptron to approximate a simple quadratic function:

$$f(x) = 2x^2 + 3x - 4$$

Write a C++ program that simulates the training process of a perceptron to fit the function above.

Steps to Solve the Problem:

1. **Initialize the Weights and Biases:** Choose arbitrary starting values for w (weight) and b (bias).
2. **Define the Target Point:** Pick a fixed input, for example, $x = 3$, and compute the true function value:

$$f(3) = 2 \cdot 3^2 + 3 \cdot 3 - 4 = 23$$

3. **Perceptron's Guess:** Use the formula:

$$y = w \cdot x + b$$

to calculate the predicted value.

4. **Calculate the Loss:** Measure the error(loss) with

$$loss = f(x) - y$$

The smaller the loss, the closer the guess is to the true value.

5. **Update the Weights and Biases:** Adjust using:

$$w = w + \alpha \cdot loss \cdot x$$

$$b = b + \alpha \cdot loss$$

where α is the learning rate (e.g., 0.01).

6. **Repeat Until Loss is Small:** Continue the process until the loss falls below a small threshold (e.g., 0.001).

```
fall25/Lab_Solutions/lab04 on 1 main
> g++ task4.cpp && ./a.out
Starting training with initial w: 0.3, b: 0.6
Target point: (3, f(3) = 23)
Press Enter to start training.
```

```
Epoch 122:
Perceptron guess: y = 22.9999
Loss: 6.24859e-05
Updated w: 6.74998, b: 2.74999

Epoch 123:
Perceptron guess: y = 22.9999
Loss: 5.62373e-05
Updated w: 6.74998, b: 2.74999

Epoch 124:
Perceptron guess: y = 22.9999
Loss: 5.06136e-05
Updated w: 6.74999, b: 2.75

Epoch 125:
Perceptron guess: y = 23
Loss: 4.55522e-05
Updated w: 6.74999, b: 2.75

Training finished after 124 epochs.
Final y: 23
Final w: 6.74999, b: 2.75
Perceptron learned the correct function!
```